

AIES — Unit I Notes

Introduction to Artificial Intelligence and Search Strategies

Table of Contents

1. Introduction to Artificial Intelligence
 2. Intelligent Agent
 - Types of Agents
 - PEAS Analysis
 3. Environment and Environment Types
 4. State Space Search
 - Search Problem Formulation
 - Evaluating Search Strategies
 5. Uninformed Search Techniques
 - Breadth-First Search (BFS)
 - Depth-First Search (DFS)
 - Iterative Deepening Search (IDS)
 6. Informed Search Techniques
 - Heuristic Function
 - A* Algorithm
 - AO* Algorithm
 7. Hill Climbing
 - Drawbacks of Hill Climbing
 - Variants of Hill Climbing
 8. Constraint Satisfaction Method
 - Backtracking Search
 - Improving Backtracking Efficiency
-

1. Introduction to Artificial Intelligence

1. Artificial Intelligence is the effort to make computers think or act like humans — the term was coined by John McCarthy in 1956 at the Dartmouth Conference.

2. AI is defined in four ways — thinking humanly, acting humanly, thinking rationally, and acting rationally — different researchers focus on different definitions.
 3. The Turing Test, proposed by Alan Turing in 1950, is a benchmark where a machine must convince a human examiner that it is human through natural language conversation.
 4. AI inherits ideas from philosophy (reasoning), mathematics (logic), psychology (understanding the mind), linguistics (language structure), and computer science (implementation).
 5. A key milestone is Deep Blue defeating chess world champion Garry Kasparov in 1997, showing AI can outperform humans in well-defined tasks.
-

2. Intelligent Agent

1. An agent is anything that perceives its environment through sensors and acts upon it through actuators to achieve a goal.
 2. A rational agent always selects the action that is expected to maximise its performance measure based on everything it has perceived so far.
 3. The agent function maps the full percept history to an action — written as $f: P^* \rightarrow A$ — and the agent program runs this function on a physical architecture.
 4. Bounded rationality means an agent cannot always be perfectly rational because of limited time, memory, and knowledge — it uses approximations to make decisions.
 5. An autonomous agent relies on its own learning and experience rather than only on knowledge built in by its designer.
-

2a. Types of Agents

Simple Reflex Agent

1. A simple reflex agent works only on the current percept — it has no memory of past states whatsoever.
2. It operates using condition-action rules — if a specific condition is detected in the current percept, a specific action is triggered.
3. It is stateless — it does not track what has happened before and treats every situation independently.
4. Example — a thermostat that turns on heating when temperature drops below a set level, without remembering previous temperatures.
5. It fails in partially observable environments because it cannot infer missing information from memory.

Model-Based Reflex Agent

1. A model-based reflex agent maintains an internal state that keeps track of the part of the world it cannot currently see.
2. It uses knowledge about how the world changes over time to update its internal state even when the environment is not fully visible.
3. The internal state allows the agent to make better decisions because it remembers what it has already perceived and done.
4. Example — a self-driving car that remembers the positions of other vehicles even when they go out of sensor range.
5. It is more powerful than a simple reflex agent because it can handle partially observable environments.

Goal-Based Agent

1. A goal-based agent has both an internal state and a description of desirable goal states it wants to reach.
2. It reasons about future consequences of its actions and selects actions that will eventually lead to the goal.
3. The agent is more flexible than reflex agents because changing the goal changes its behaviour without rewriting all the rules.
4. Example — a navigation system that plans a route to a destination by considering multiple possible paths and choosing the one that reaches the goal.
5. It is less efficient than reflex agents for immediate reactions but much better at complex multi-step planning.

Utility-Based Agent

1. A utility-based agent uses a utility function that maps each state to a real number representing how desirable that state is.
2. It can handle competing goals and tradeoffs by selecting the action that maximises expected utility rather than just reaching any goal.
3. When multiple goals conflict, the utility function helps the agent decide which goal is more important based on numeric values.
4. Example — a medical diagnosis system that balances treatment effectiveness, cost, and side effects using a utility score for each option.
5. Utility-based agents are the most sophisticated type because they can reason about uncertainty and make principled tradeoffs.

Learning Agent

1. A learning agent has four components — a performance element (the current agent function), a learning element (makes improvements), a critic (gives feedback), and a problem generator (suggests new experiences).
 2. The critic evaluates how well the agent is performing based on a fixed performance standard and sends feedback to the learning element.
 3. The learning element uses the critic's feedback to modify the performance element so that future decisions are better.
 4. The problem generator suggests actions that lead to new informative experiences, encouraging the agent to explore rather than always exploit known strategies.
 5. Learning agents improve over time — they start with basic knowledge and get better with experience, unlike fixed-rule agents.
-

2b. PEAS Analysis

1. PEAS stands for Performance Measure, Environment, Actuators, and Sensors — it is used to fully specify the task environment of any rational agent.
2. **Performance Measure** defines the criteria used to judge how well the agent is doing — for example, safe driving, speed, fuel efficiency, and passenger comfort for a self-driving car.
3. **Environment** describes the physical and social context the agent operates in — for a self-driving car this includes roads, traffic, pedestrians, and weather conditions.
4. **Actuators** are the means through which the agent acts on the environment — for a self-driving car these are the steering wheel, accelerator, brakes, and horn.
5. **Sensors** are the means through which the agent perceives the environment — for a self-driving car these include cameras, GPS, radar, speedometer, and microphones.

Example — Part-Picking Robot:

PEAS Component	Description
Performance	Correctly identified parts, speed, minimised errors
Environment	Conveyor belt, parts of different sizes and shapes
Actuators	Robotic arm, gripper, camera positioning system
Sensors	Camera, touch sensors, proximity sensors

3. Environment and Environment Types

1. The type of environment largely determines what kind of agent design is most appropriate — a more complex environment needs a more sophisticated agent.
2. A **fully observable** environment allows the agent's sensors to detect the complete state at all times — chess is fully observable because the entire board is visible.
3. A **deterministic** environment means the next state is completely determined by the current state and the agent's action — a **stochastic** environment has randomness and uncertainty.
4. An **episodic** environment means each episode is independent — decisions in one episode do not affect future episodes — while a **sequential** environment means current actions affect all future states.
5. A **static** environment does not change while the agent is thinking — a **dynamic** environment keeps changing, requiring the agent to act quickly.

Property	Type A	Type B	Example
Observability	Fully Observable	Partially Observable	Chess vs Poker
Outcome	Deterministic	Stochastic	Puzzle vs Self-driving car
Episodes	Episodic	Sequential	Image classification vs Chess
Change	Static	Dynamic	Crossword vs Soccer
States	Discrete	Continuous	Chess moves vs Robot arm
Agents	Single-Agent	Multi-Agent	Puzzle vs Chess

4. State Space Search

1. State space search is a problem-solving method where the problem is represented as a set of states, and the goal is to find a sequence of actions that transforms the initial state into a goal state.
2. A **state** is defined by all the relevant attributes of the world at a given point in time — for example, the position of all tiles in the 8-puzzle.
3. An **operator** is an action that transforms one state into another — it has a precondition that must be true before it can be applied.
4. A search problem is formally represented as a 4-tuple $\{S, s_0, A, G\}$ — S is the full set of states, s_0 is the initial state, A is the set of operators, and G is the set of goal states.

5. A **solution** is a sequence of actions $P = \{a_0, a_1, \dots, a_N\}$ that leads from the initial state through intermediate states to a goal state.
-

4a. Search Problem Formulation

1. A search problem is represented as a directed graph where states are nodes and legal actions between states are arcs connecting those nodes.
 2. A search tree node contains a state description, a pointer to its parent node, the depth of the node, the operator that generated it, and the path cost from the start.
 3. The **OPEN list** (also called the fringe) keeps track of all nodes that have been generated but not yet explored.
 4. The **CLOSED list** records all nodes that have already been expanded so they are not revisited.
 5. The **branching factor b** is the maximum number of children any non-leaf node can have — it directly determines the size and complexity of the search tree.
-

4b. Evaluating Search Strategies

1. **Completeness** — will the strategy always find a solution if one exists anywhere in the state space?
 2. **Optimality** — will the strategy always find the solution with the lowest path cost among all possible solutions?
 3. **Time Complexity** — how many nodes does the algorithm expand in the worst case, measured using b (branching factor), d (depth of shallowest solution), and m (maximum depth)?
 4. **Space Complexity** — how much memory does the algorithm need, measured as the maximum size of the fringe at any point during the search?
 5. In practice, space complexity is often more limiting than time complexity because running out of memory stops the search entirely.
-

5. Uninformed Search Techniques

Uninformed (blind) search strategies have no extra knowledge about the state space — they cannot estimate how close a state is to the goal.

Strategy	Complete	Optimal	Time	Space
BFS	Yes	Yes (uniform cost)	$O(b^d)$	$O(b^d)$
DFS	Yes (finite)	No	$O(b^m)$	$O(bm)$
IDS	Yes	Yes (uniform cost)	$O(b^d)$	$O(bd)$

5a. Breadth-First Search (BFS)

1. BFS explores the search tree level by level — it expands all nodes at the current depth before moving to nodes at the next depth.
2. BFS is implemented using a **FIFO queue** — newly generated nodes are added to the back so shallower nodes are always expanded first.
3. BFS is **complete** — it will always find a solution if one exists because it exhaustively searches every level before going deeper.
4. BFS is **optimal** when all actions have equal cost because it always finds the shallowest solution first which is also the cheapest.
5. The main disadvantage is very high memory usage — at depth d with branching factor b , both time and space complexity are $O(b^d)$ which grows exponentially.

5b. Depth-First Search (DFS)

1. DFS explores the tree by following each path to its greatest depth before backtracking and trying another path.
2. DFS is implemented using a **LIFO stack** — newly generated nodes are added to the front so the deepest node is always expanded first.
3. DFS requires very little memory — it only stores the nodes on the current path from root to the current node, giving $O(bm)$ space complexity.
4. DFS is **not optimal** — it may find a longer or costlier solution first since it goes deep before considering alternatives.
5. DFS may get stuck in infinite loops if the state space contains cycles — it does not guarantee finding a solution in infinite search spaces.

5c. Iterative Deepening Search (IDS)

1. IDS is a combination of DFS and BFS — it gradually increases the depth limit from 0 upward until the goal is found.
 2. At each iteration, a depth-limited DFS is performed with the current depth limit — if no solution is found, the depth limit increases by one and the search restarts.
 3. IDS combines the **memory efficiency of DFS** (space $O(bd)$) with the **completeness and optimality of BFS** for uniform cost problems.
 4. Although IDS regenerates states from earlier iterations, it is faster than BFS in practice because BFS must store all nodes up to level $d+1$ in memory.
 5. IDS is the **preferred uninformed search strategy** when the search space is large and the depth of the goal node is unknown in advance.
-

6. Informed Search Techniques

Informed search algorithms use a **heuristic function** to guide the search toward the goal more efficiently, reducing the number of nodes explored.

6a. Heuristic Function

1. A heuristic function **$h(n)$** provides an estimate of the cost of the cheapest path from the current node n to a goal node.
 2. A heuristic is **admissible** if it never overestimates the true cost to reach the goal — $h(n) \leq h^*(n)$ where $h^*(n)$ is the actual optimal cost.
 3. For the **8-puzzle problem**, two common heuristics are the Hamming distance (number of misplaced tiles) and the Manhattan distance (sum of individual tile distances to their goal positions).
 4. The **Manhattan distance** is a better heuristic than Hamming distance because it gives a tighter estimate — it is closer to the actual cost without overestimating.
 5. The quality of the heuristic directly affects search efficiency — a better heuristic expands fewer nodes while still guaranteeing an optimal solution.
-

6b. A* Algorithm

1. A* is the most widely used informed search algorithm — it evaluates each node using **$f(n) = g(n) + h(n)$** , where $g(n)$ is the actual cost from start to n and $h(n)$ is the admissible heuristic estimate from n to the goal.

2. A* always expands the node with the **lowest $f(n)$** value — it balances the cost already paid and the estimated cost remaining.
3. A* is **complete and optimal** as long as the branching factor is finite, step costs are non-zero, and the heuristic is admissible (never overestimates).
4. The main drawback of A* is its **high memory requirement** — it keeps all generated nodes in memory which makes it impractical for very large problems.
5. Time and space complexity are both $O(b^d)$ but with a good heuristic, A* expands far fewer nodes than uninformed search in practice.

*A Step-by-Step Table Format (Romania Map — Arad to Bucharest):**

Step	Node Expanded	$g(n)$	$h(n)$	$f(n)$
1	Arad	0	366	366
2	Sibiu	140	253	393
3	Rimnicu Vilcea	220	193	413
4	Pitesti	317	100	417
5	Bucharest	418	0	418 ✓

Path: Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest (Total cost = 418)

6c. AO* Algorithm

1. AO* is designed for **AND-OR graphs** where solving a problem may require solving ALL subproblems (AND node) or just ONE subproblem (OR node).
2. The algorithm maintains two lists — **OPEN** (nodes traversed but not yet marked solvable or unsolvable) and **CLOSED** (nodes already processed).
3. AO* works by computing the most promising solution tree, selecting an unexpanded node from it, expanding that node, and propagating updated costs backward.
4. A node is marked **solved** when it is a terminal goal and all its ancestors are also solved — the algorithm succeeds when the starting node is marked solved.
5. AO* is **optimal** and handles both AND and OR arcs — but it may fail for unsolvable nodes where no path to a goal exists.

*AO Algorithm Steps:**

1. Place starting node into OPEN.

2. Compute the most promising solution tree T_0 .
 3. Select a node n from OPEN that is in T_0 — move it to CLOSED.
 4. If n is a terminal goal, mark n as solved and propagate to ancestors — exit if start node is solved.
 5. If n is unsolvable, mark it and propagate — exit with failure if start node is unsolvable.
 6. Expand n , compute $h(n)$ for all successors, push them into OPEN.
 7. Return to Step 2.
-

7. Hill Climbing

1. Hill climbing is a **local search algorithm** that continuously moves to the neighbouring state with the highest objective function value until no neighbour is better.
 2. It is called a **greedy local search** because it only looks at immediate neighbours and never backtracks — it makes only locally optimal choices at each step.
 3. A node in hill climbing only stores the **current state and its value** — it does not build a search tree or remember the history of states visited.
 4. Hill climbing is used for **optimisation problems** like the Travelling Salesman Problem where the goal is to incrementally improve a current solution.
 5. Memory requirement is **constant** — because it never stores the search tree, memory usage does not grow regardless of how large the state space is.
-

7a. Drawbacks of Hill Climbing

Local Maximum

1. A local maximum is a state that is better than all its immediate neighbours but worse than the global maximum.
2. The algorithm gets stuck here because it cannot move to any better neighbour — it has no way to know whether a better solution exists elsewhere.
3. **Solution** — Use backtracking to maintain a list of promising paths so the algorithm can return and try alternative directions when stuck.

Plateau

1. A plateau is a flat region of the state space where all neighbouring states have the same objective function value.
2. The algorithm loses direction on a plateau because no neighbour is strictly better — it does not know which way to move.

3. **Solution** — Take a random jump to a state far away from the current position to escape the plateau and potentially reach a higher region.

Ridge

1. A ridge is a special type of local maximum that has a sloped surface higher than its surroundings but cannot be reached in a single uphill move.
 2. The algorithm cannot navigate a ridge because every individual step appears to go downhill even though the ridge itself is higher.
 3. **Solution** — Use bidirectional search or allow moves in multiple directions simultaneously to navigate ridges more effectively.
-

7b. Variants of Hill Climbing

Stochastic Hill Climbing

1. Instead of always choosing the best uphill neighbour, stochastic hill climbing **randomly selects** from among the available uphill moves.
2. This randomness helps avoid getting stuck at ridges since different uphill paths are explored rather than always the steepest one.
3. It is less deterministic than simple hill climbing but can sometimes find better solutions by avoiding suboptimal greedy choices.
4. The tradeoff is that it may take longer to converge since it does not always select the best available move at each step.
5. It is particularly useful when the landscape has many ridges or multiple local maxima with similar values.

First Choice Hill Climbing

1. First choice hill climbing generates successor states **randomly one at a time** and immediately moves to the first one that is better than the current state.
2. It does not evaluate all successors before choosing — it picks the very first improvement found, making it much faster when the branching factor is very large.
3. This approach is useful in problems with thousands of possible successors where evaluating all of them before picking the best would be too slow.
4. It is a practical middle ground between simple hill climbing and stochastic hill climbing.
5. It can escape some local maxima faster than simple hill climbing because it acts on the first improvement rather than spending time comparing all neighbours.

Random Restart Hill Climbing

1. Random restart hill climbing addresses the local maximum problem by **randomly generating a new initial state** and restarting the search multiple times.
 2. Each restart begins from a different region of the state space, giving the algorithm a chance to discover the global maximum.
 3. When no more progress can be made from the current state, the algorithm discards it and begins fresh from a randomly chosen position.
 4. The success of this method depends on the **percentage of the state space that represents goal states** — more goal states mean faster success.
 5. This is one of the most practical improvements to basic hill climbing for problems with many local maxima.
-

8. Constraint Satisfaction Method

1. A **Constraint Satisfaction Problem (CSP)** is defined by three components — a set of **variables X**, a set of **domains D** (possible values for each variable), and a set of **constraints C** that specify allowable combinations.
 2. The goal is to find an assignment of values to all variables such that **all constraints are simultaneously satisfied** — called a consistent and complete assignment.
 3. Constraints can be **unary** (involving one variable), **binary** (involving two variables), or **higher-order** (involving three or more variables).
 4. The **map-coloring problem** is a classic CSP — variables are regions, domains are colors (red, green, blue), and constraints require adjacent regions to have different colors.
 5. CSP methods are more powerful than standard search because constraint information can be used to **prune the search space early**, preventing invalid assignments from being explored.
-

8a. Backtracking Search

1. Backtracking search is a **depth-first search for CSPs** — it assigns one variable at a time and backtracks when no legal value remains for any variable.
2. Variable assignments are **commutative** — the order of assignment does not affect the final solution, so only one variable is assigned at each node.
3. The algorithm selects a variable, assigns it a value, checks all constraints against previously assigned variables, and either proceeds or backtracks.

4. If a variable is assigned all its values without finding a consistent one, the algorithm **pops back** to the previous variable and tries the next available value.
 5. Backtracking can solve the n-queens problem for up to approximately $n = 25$ queens but becomes very slow for larger or more complex constraint problems.
-

8b. Improving Backtracking Efficiency

Most Constrained Variable (MRV) Heuristic

1. The MRV heuristic selects the variable with the **fewest remaining legal values** as the next variable to assign.
2. Choosing the most constrained variable first allows the algorithm to detect failures early and reduces the branching factor.
3. This prevents the algorithm from exploring subtrees that will inevitably fail due to constraint violations on the most restricted variable.
4. If two variables have the same number of remaining values, the **most constraining variable** is used as a tiebreaker — the variable with the most constraints on other unassigned variables.
5. Combining MRV with the least-constraining value heuristic (choosing the value that rules out fewest options for others) can make even 1000-queens problems feasible.

Forward Checking

1. Forward checking tracks the **remaining legal values for all unassigned variables** after each new assignment and eliminates values that would violate a constraint.
 2. The search terminates early when any unassigned variable's domain becomes **empty**, avoiding exploration of states that will definitely fail.
 3. Forward checking propagates the implications of the current assignment to neighbouring variables, providing early detection of inevitable failures.
 4. It is more powerful than simple backtracking because it checks constraints **ahead of time** rather than waiting until a variable is actually being assigned.
 5. Constraint propagation goes further than forward checking by enforcing **arc consistency** — ensuring every value in every domain is consistent with all constraints in the graph.
-